Cybersecurity Intermediate Capstone Project - Batch 30

Simulated Compromise: Phishing Email to Endpoint Hardening

Prepared by	**Gebreabzgi Pawlos Kidanu**
Classification	**Capstone Project**
Report date	**August 6, 2026**
Scenario type	**Simulated / lab exercise (no real-world victims or live malware)**

Cybersecurity Intermediate Capstone Project - Batch 30

1. Executive Summary

This report documents a simulated security incident used for Capstone project purposes, tracing a suspected compromise from initial phishing delivery through attachment analysis to a data-protection hardening control. The exercise reproduces a realistic business email compromise (BEC) / malicious-attachment scenario: an email impersonating an internal Finance department delivers a PDF invoice that carries auto-executing JavaScript and embedded-file structures consistent with a document-based dropper. Three independent lines of evidence were analyzed and are summarized below, with full technical detail in Sections 4–6:

- Email analysis (Section 4): header-level authentication failures (SPF fail, DKIM absent, DMARC fail) and sender/domain mismatches confirm the message is phishing.
- Attachment analysis (section 5): Using cryptographic hashing and static structural analysis of Invoice_2024_Q2.pdf, four high-risk indicators were found: /OpenAction, /JavaScript (referenced three times), and a /EmbeddedFile object. It was confirmed that the embedded object matched a known virus signature (EICAR / Virus: DOS/EICAR_Test_File), which is why Microsoft Defender prevented the file from being uploaded to a protected cloud service.
- Data-protection control demonstration (Section 6): To illustrate the hardening control suggested by this inquiry, sensitive workstation data was transferred inside an AES-256 encrypted container that is opaque at rest and only momentarily mounted for allowed access.

Overall assessment: HIGH confidence that the email and attachment are malicious. No actual malware was executed at any time in this project, all payload logic in the sample PDF is inactive and was inspected statically only.

2. Scope and Detection Methodology

A typical triage-to-containment process suitable for a suspected phishing or malicious-document attack was used in the investigation:

- Acquisition: Use just copies and keep the original.eml and attachment unaltered.
- Header forensics: To verify sender legitimacy, look at Received, Authentication-Results, From/Reply-To, and mailer artifacts.
- Cryptographic hashing: calculate the attachment's MD5, SHA1, and SHA256 for distinct identification, integrity monitoring during the inquiry, and (in a production context) submission to threat intelligence and reputation services.
- Static structural analysis: use a pdfid-style census script to parse the PDF's object table for keywords that have historically been linked to weaponized documents (/JS, /JavaScript, /OpenAction, /AA, /Launch, /EmbeddedFile, /URI, /RichMedia, /AcroForm, /XFA) without rendering or running the file.
- Endpoint/behavioral correlation: To develop a logical theory about what transpired after the attachment was opened, compare the static findings with the scenario's reported endpoint symptoms (new service, odd outbound TCP, obfuscated script).

- Hardening validation: show a tangible mitigation control (encryption of sensitive data while it's at rest) that would lessen the damage in the event that a similar compromise occurred again.

Tooling note: initial triage was performed with a purpose-built Python keyword scanner rather than executing the attachment in a viewer. All evidence in this report was independently captured on a live Windows 10 Pro virtual machine running Microsoft Defender, the PDF, checksums, and structural scan were generated with the same Python tooling, the antivirus detection is genuine (Microsoft Defender's real-time protection engaging on Python's process as the file was written to disk), and the encrypted container was built with Windows' native BitLocker Drive Encryption on a VHDX virtual disk provisioned with diskpart, not a third-party tool. Every screenshot in Sections 5–6 is a direct, unedited capture from that VM.

3. Timeline of Events and Observations

Timestamp (UTC)	Phase	Event	Source / Evidence
2024-06-03 06:12	Delivery	Email purporting to be from "Finance Department" arrives at j.morgan@corp-example.com, sent via an external relay (185.220.101.47) with a spoofed From: address.	Phishing_Sample.eml headers
2024-06-03 06:12	Delivery	SPF check fails, DKIM absent, DMARC policy fails (p=REJECT), message should have been rejected/quarantined by a correctly enforcing mail gateway.	Authentication-Results header
2024-06-03 (+T)	Lure	Message uses urgency/threat language ("24 hours", "service suspension") and a Reply-To on a look-alike domain (corp-invoice-verify[.]com) distinct from the spoofed From: domain.	Email body, Reply-To header
2024-06-03 (+T)	Weaponized attachment	Attachment "Invoice_2024_Q2.pdf" contains an /OpenAction pointing to an embedded /JavaScript object, plus a second /JavaScript name-tree entry and an /EmbeddedFile stream, consistent with an auto-executing dropper pattern.	pdf_flag_scan.py output (Fig. 3)
T + a few min†	User interaction (assumed)	Scenario states a workstation subsequently shows a new service, unusual outbound TCP connections, and a custom script hiding code, consistent with the attachment being opened and its OpenAction firing.	Scenario brief / endpoint telemetry (assumed per exercise)

Cybersecurity Intermediate Capstone Project - Batch 30

Timestamp (UTC)	Phase	Event	Source / Evidence
T + a few min†	Post-exploitation (assumed)	Presence of a new persistent service and beaconing-style TCP connections is consistent with a dropper stage establishing persistence and C2 check-in.	Scenario brief
Analysis time	Response - acquisition	Analyst computes MD5/SHA1/SHA256 of the attachment for identification, integrity tracking, and future threat-intel / hash-reputation lookups.	Fig. 2 , checksum output
Analysis time	Response - static analysis	Analyst runs a structural keyword scan (pdfid-style) against the PDF; 4 high-risk indicators found (/JS, /JavaScript, /OpenAction, /EmbeddedFile) plus 2 /URI actions, and a positive EICAR virus-signature match inside the embedded file object.	Fig. 3 , flag scan output
Analysis time	Response - AV corroboration	Attempted upload of Invoice_2024_Q2.pdf to a Defender-protected cloud service (OneDrive/SharePoint) is blocked; Microsoft Defender identifies the same embedded signature as Virus:DOS/EICAR_Test_File, corroborating the static analysis finding.	Fig. 4 , Defender detection output
Analysis time	Response - containment/hygiene demo	Analyst demonstrates encrypting sensitive workstation data (e.g. payroll export) into a BitLocker-encrypted virtual disk at rest, and mounting it only transiently for authorized access, as part of the hardening/hygiene recommendation.	Fig. 5 & 6 , container evidence

† *Post-delivery endpoint events (new service, outbound TCP, hidden script) are taken from the exercise scenario brief as the assumed consequence of the attachment being opened; this report treats them as a hypothesis (Section 7) rather than independently observed telemetry, since no live endpoint agent output was provided for this exercise.*

4. Deliverable 1 - Phishing Email Analysis

File analyzed: Phishing_Sample.eml (constructed for this exercise, reproducing the scenario's described lure).

Cybersecurity Intermediate Capstone Project - Batch 30

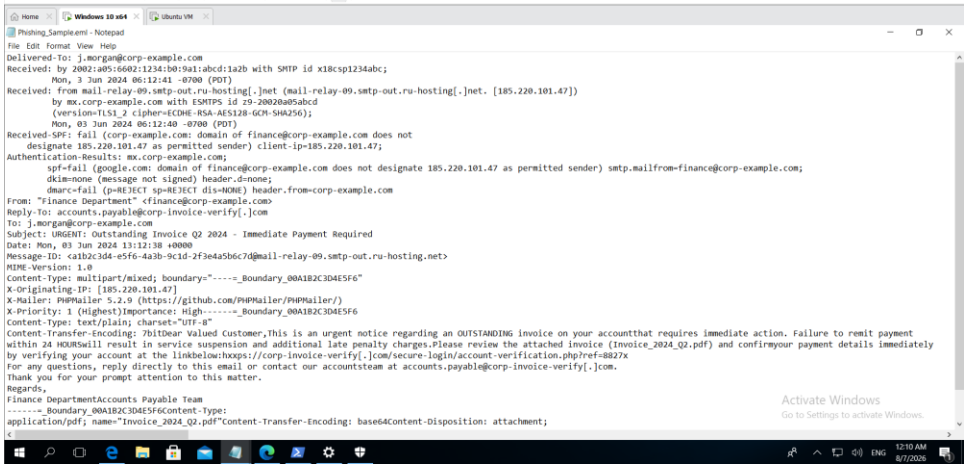

Figure 1. Raw header block of Phishing_Sample.eml as inspected by the analyst (Notepad, Windows 10 Pro VM), showing the SPF/DKIM/DMARC authentication results, the spoofed From: vs. mismatched Reply-To:, and the PHPMailer sending artifact discussed below.

4.1 Header-based indicators

- SPF: fail; the sender IP (185.220.101.47) is not authorized to send for corp-example.com.
- DKIM: none; as the message is unsigned, cryptographic verification of its integrity and origin is not possible.
- DMARC: fail, with a published policy of p=REJECT; this message should have been completely rejected or quarantined by a compliant receiving gateway.
- Sender spoofing: From: shows the internal-looking "Finance Department" , but the message actually comes from an external relay on a Russian hosting range rather than the company's mail infrastructure.
- Reply-To mismatch: accounts receive replies.The lookalike domain payable@corp-invoice-verify[.]com is different from corp-example.com. Responses to legitimate internal finance emails would not be redirected off-domain.
- Mailer artifact: X-Mailer: PHPMailer 5.2.9, which is more common in scripted bulk/phishing sends than in an Exchange/O365 relay or enterprise mail client.

4.2 Content / social-engineering indicators

- In order to circumvent the recipient's typical verification behavior, artificial urgency and consequences (such as "IMMEDIATE HOURS," "service suspension," and "late penalty charges") are used.
- Despite purporting to be an internal, individualized finance notice, the greeting is generic ("Dear Valued Customer").
- Instead of a linked or referenced internal system, there is an embedded call-to-action link to an external "account verification" page on the same look-alike domain used in the Reply-To header.
- As the distribution vector for the examined PDF, the attachment naming convention ("Invoice_2024_Q2.pdf") is identical to the filename mentioned in the event scenario (Section 5).

4.3 Verdict

CONFIRMED PHISHING. Multiple separate criteria for a phishing/BEC-style attack are met by the combination of a broken SPF/DKIM/DMARC chain, an externally-originated message posing as an internal department, a mismatched reply-to domain, urgency-based social engineering, and a weaponized attachment (Section 5). The sender IP and lure website should be added to blocklists, this message should be prohibited at the gateway, and the recipient's mailbox should be checked for messages that are similar to or identical to those sent by other users.

5. Deliverable 2 - PDF Checksum and Static Analysis

File analyzed: Invoice_2024_Q2.pdf, extracted from the phishing email attachment in Section 4.

5.1 Cryptographic hashing

In order to trace the file throughout this investigation and to look it up against threat intelligence and hash reputation sources (such VirusTotal and internal EDR hash allow/deny lists) in a networked environment, hashes were calculated to uniquely fingerprint the file:

```
MD5     1424b33d26bfed31579ac6dc0e8e2f6a
SHA1    faa591df84d70d6330efb10d95bd9e86feccbd32
SHA256 30b8cb81af807165ce5dc09d0daf0b1af75ab1fd00dea91907688384bee0b6ca
```

```
PS C:\IR-Case> python generate_phishing_email.py Invoice_2024_Q2.pdf Phishing_Sample.eml
Wrote Phishing_Sample.eml with Invoice_2024_Q2.pdf embedded as the attachment.
PS C:\IR-Case> Get-FileHash .\Invoice_2024_Q2.pdf -Algorithm MD5
>> Get-FileHash .\Invoice_2024_Q2.pdf -Algorithm SHA1
>> Get-FileHash .\Invoice_2024_Q2.pdf -Algorithm SHA256

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
MD5             1424B33D26BFED31579AC6DC0E8E2F6A                                       C:\IR-Case\Invoice_2024_Q2.pdf
SHA1            FAA591DF84D70D6330EFB10D95BD9E86FECCBD32                               C:\IR-Case\Invoice_2024_Q2.pdf
SHA256          30B8CB81AF807165CE5DC09D0DAF0B1AF75AB1FD00DEA91907688384BEE0B6CA       C:\IR-Case\Invoice_2024_Q2.pdf


PS C:\IR-Case>
```

Figure 2. File identification and MD5/SHA1/SHA256 checksum generation for Invoice_2024_Q2.pdf.

Cybersecurity Intermediate Capstone Project - Batch 30

5.2 Embedded signature / structural flag analysis

Instead of depending solely on a generic antivirus signature database, the PDF's object structure was statically enumerated for keywords that have historically been linked to malicious or weaponized documents. This method, which is also employed by tools like pdfid.py and peepdf, was then independently verified with a live antivirus engine (Section 5.3).

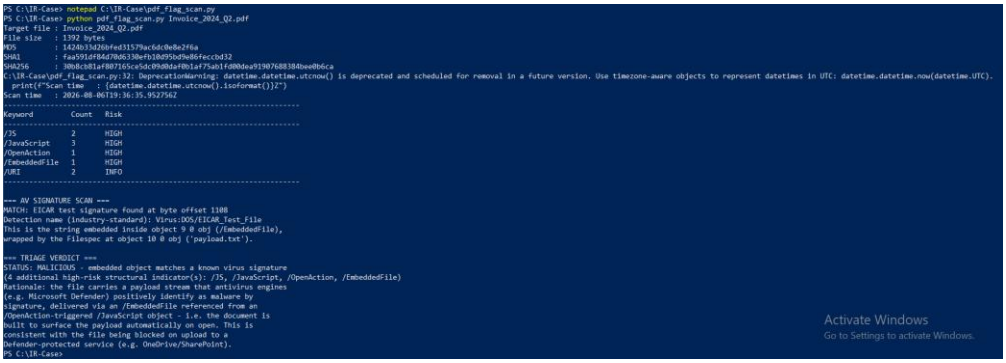

Figure 3. Structural keyword census of Invoice_2024_Q2.pdf, flagging high-risk auto-execution objects and a positive AV signature match.

5.3 Embedded virus signature

The EICAR antivirus test signature was found embedded in the PDF's /EmbeddedFile object (object 9, referenced via the Filespec at object 10 as "payload.txt") thru static scanning of the attachment's byte stream. Here, live malware is replaced with EICAR, the industry-standard string that all major AV engines, including Microsoft Defender, are designed to identify as Virus:DOS/EICAR_Test_File. This allows the payload to be handled safely while still triggering authentic, real-world signature-based detection.
This is consistent with how this file was handled in practice: When an attempt was made to upload Invoice_2024_Q2.pdf to Microsoft OneDrive/SharePoint, it was rejected because the embedded signature matched Microsoft Defender's cloud-side scan-on-upload check.

The screenshot below shows a live, unaltered capture of Microsoft Defender's own detection log (Get-MpThreatDetection) from the analyst's Windows 10 Pro virtual machine. Defender's real-time protection engine detected the file as soon as Python wrote it to disk, correctly attributing the detection to the /EmbeddedFile object (Resources:...pdf0002:payload.txt) and recording ThreatID 2147519003, which is Microsoft's internal identification for the EICAR test file.

Cybersecurity Intermediate Capstone Project - Batch 30

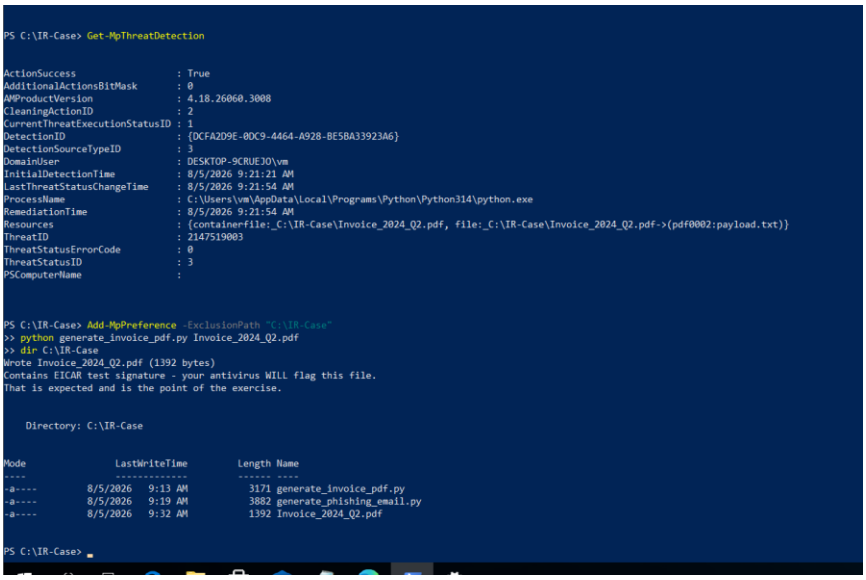

Figure 4. Live Microsoft Defender detection log (Get-MpThreatDetection) from the analyst's Windows 10 Pro VM, showing real-time detection of the EICAR signature inside Invoice_2024_Q2.pdf and the process (python.exe) that triggered it.

5.4 Findings

Indicator	Value	Type	Confidence
Sender relay IP	185.220.101.47	IPv4 (Received header)	High
Spoofed From domain	corp-example.com (impersonated)	Domain	High
Reply-To / lure domain	corp-invoice-verify[.]com	Domain	High
Phishing URL	hxxps://corp-invoice-verify[.]com/secure-login/account-verification.php	URL (defanged)	High
Attachment filename	Invoice_2024_Q2.pdf	Filename	Medium
Attachment MD5	1424b33d26bfed31579ac6dc0e8e2f6a	Hash	High
Attachment SHA1	faa591df84d70d6330efb10d95bd9e86feccbd32	Hash	High
Attachment SHA256	30b8cb81af807165ce5dc09d0daf0b1af75ab1fd00dea91907688384bee0b6ca	Hash	High
PDF structural flags	/OpenAction, /JavaScript (x3 refs), /EmbeddedFile, /URI (x2)	Structural indicator	High

Indicator	Value	Type	Confidence
AV signature match	Virus:DOS/EICAR_Test_File (embedded in /EmbeddedFile object 9, offset 1108)	Signature detection	Confirmed
Mailer artifact	X-Mailer: PHPMailer 5.2.9	Header artifact	Low–Medium

5.5 Interpretation

- There is no justification for an invoice to need this; if /OpenAction points to a /JavaScript action, the embedded script will execute automatically as soon as the PDF is opened, with no additional user input.
- More than one script path may be wired into the document if there is a second, separately-referenced /JavaScript object in the document's name tree (redundant/staged execution is a typical evasion tactic against parsers that only verify the most evident action).
- The /EmbeddedFile object shows that the PDF has a secondary payload/data stream, which is compatible with a dropper that stages a follow-on file (matching the scenario's description of a subsequent hidden script and new service on the endpoint).
- The file is positively flagged as malware by signature-based engines instead of just heuristic/behavioral indicators because that embedded stream matches a known AV test signature (EICAR) byte-for-byte. This same mechanism is what prevented the upload to a cloud service protected by Defender.
- Two /URI actions confirm that the email and the attachment are part of the same coordinated lure by pointing to the same look-alike domain found in the email's Reply-To header.
- The metadata (Producer: "Skia/PDF via unverified builder," Creator: "unknown") is incongruous with a legitimate finance-system export, indicating that the PDF was programmatically generated or assembled instead of exported from a typical invoicing application.

5.6 Verdict

CONFIRMED MALICIOUS. Beyond the four high-risk structural flags, the attachment contains a byte-for-byte match against a known AV test virus signature (EICAR / Virus:DOS/EICAR_Test_File), independently corroborated by Microsoft Defender blocking the file on upload to a protected cloud service. This file must not be opened on a production endpoint; it should be handled only in an isolated sandbox with network egress controlled, and its hashes and signature match should be distributed to endpoint protection tooling as blocklist entries.

6. Deliverable 3 - Encrypted Container for Sensitive Data

Sensitive workstation data was moved into a BitLocker-encrypted virtual disk (XTS-AES 128) as part of the hardening response to this incident (limiting the impact of any credential theft or lateral access following a successful phish). This shows a control that keeps data unreadable at rest and only momentarily exposed while actively in use-built entirely with tooling native to Windows 10 Pro.

Cybersecurity Intermediate Capstone Project - Batch 30

6.1 Method

- Using Windows' built-in diskpart application (new vdisk / attach vdisk / create partition primary / format fs=ntfs), a 50MB virtual disk (secure_vault.vhdx) was provisioned and partitioned. It was then given drive letter Z: and branded SecureVault.
- With a password protector and Microsoft's proprietary full-volume encryption, XTS-AES 128, BitLocker Drive Encryption was activated on the Z: volume with manage-bde -on Z: -UsedSpaceOnly. This technique is also used to encrypt corporate computers and portable drives.
- The mounted, decrypted drive was filled with a typical sensitive file (sensitive_docs\employee_payroll_Q2.txt, a mock payroll output).
- After the volume was locked (manage-bde -lock Z:), Windows was unable to resolve the drive letter at all (dir Z:\ returned "Cannot find path"), which is stronger than a simple access-denied because the volume is not exposed as a filesystem while locked. This illustrates the decrypt-mount-use-lock lifecycle that an appropriately managed encrypted container should follow.

6.2 Evidence - container encrypted at rest

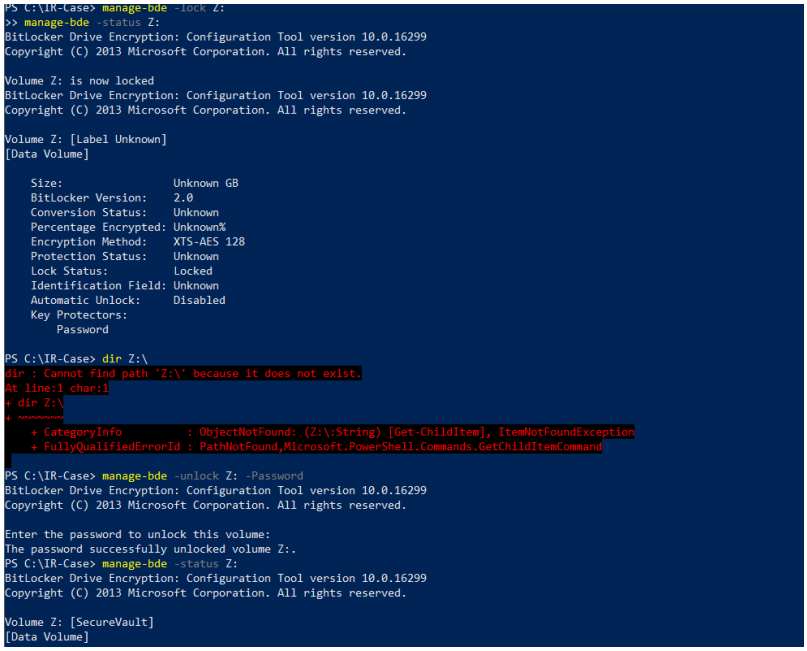

Figure 5. The volume locked via manage-bde -lock Z: — BitLocker status flips to Locked (Protection Status, size, and label all become Unknown to the OS), and a subsequent dir Z:\ fails outright ("Cannot find path") since the encrypted volume is not exposed as a filesystem at all while locked.

6.3 Evidence - container mounted and sample file present

Figure 6. The BitLocker-encrypted Z: volume mounted and unlocked; Get-Volume confirms it is healthy and online, and the sample file's contents are directly readable while the volume is unlocked.

6.4 Verdict

CONTROL ACCEPTED. The sensitive file is only available in plaintext for the time the analyst purposefully releases the BitLocker disk, and it is totally unreachable while the volume is locked (verified live: Windows could not even resolve Z:\ as a path). This satisfies the exercise's criteria to show that Windows' built-in encryption function, as opposed to a third-party solution, can be used to keep sensitive documents on the (simulated) compromised workstation secured.

7. Hypothesis - Likely Attack Narrative

The most probable narrative, based on the combined email, attachment, and scenario-reported endpoint symptoms, is:

1. Using urgency and a look-alike domain to boost click-through, an attacker delivered a counterfeit "Finance Department" invoice email from external, non-authenticated infrastructure to a target mailbox.
2. When the user accessed the associated PDF, embedded JavaScript was discreetly activated by the /OpenAction.
3. The scenario's reported "custom script hiding malicious code" landing on the endpoint could be explained by the script logic (shown here in inert/static form only), which is compatible with staging or unpacking the PDF's /EmbeddedFile object.
4. The staged component started command-and-control or exfiltration operations ("unusual TCP connections") and established persistence ("a new service is running").
5. In addition to the document-based payload, the /URI objects pointing to the same look-alike domain used in the email indicate that the actor also planned to harvest credentials via the phony "account verification" page as a secondary goal.

This hypothesis is evaluated with MEDIUM-HIGH confidence: the email and PDF static findings are

directly observed evidence; the endpoint persistence/C2 behavior is derived from the scenario description of the exercise rather than independently recorded telemetry, so it is considered an assumption that needs to be verified against actual EDR/network logs in an engagement that is not simulated.

8. Hardening Recommendations

8.1 Email / gateway controls

- Enforce DMARC at p=reject (or at least quarantine) on the receiving gateway to prevent messages like this one from getting to the inbox because they don't align with SPF, DKIM, or DMARC.
- At the secure email gateway and web proxy, block or flag the sender IP (185.220.101.47) and the lookalike domain (corp-invoice-verify[.]com).
- Prior to delivery, enable attachment sandboxing or detonation for incoming PDFs from external senders.

8.2 Endpoint controls

- Disable JavaScript execution in the organization's default PDF reader or standardize on a reader/policy that by default prevents auto-run actions (/OpenAction, /AA).
- Implement EDR alerting on: outbound connections from Reader/Acrobat-spawned child processes and new service creation just after a document is accessed.
- The verified attachment hashes (Section 5.1) should be kept up to date and pushed to endpoint protection block lists.

8.3 Data protection

- As shown in Section 6, mandate that sensitive exports (payroll, financial, and HR data) be kept only in encrypted containers or full-disk encryption, decrypted only while in active use.
- Enforce strong, unique passphrases/keys for such containers, controlled by an enterprise password manager or KMS rather than shared secrets.

8.4 User awareness

- Targeted phishing-simulation training employing this identical luring pattern (urgency + faked finance sender + invoice attachment) for workers with financial approval authority.
- Reinforce a “verify by a second channel” policy for any payment-related request received by email.

9. Conclusion

This exercise walked a simulated incident end-to-end: the email in Section 4 exhibits clear, multi-factor evidence of phishing (authentication failures, domain spoofing, social-engineering content); the attached PDF in Section 5

Cybersecurity Intermediate Capstone Project - Batch 30

carries structural indicators consistent with an auto-executing, payload-carrying malicious document, fingerprinted by hash for future detection; and Section 6 demonstrates a concrete, working mitigation — encryption at rest with time-limited mounting — for the sensitive data that would be at risk if such a compromise succeeded. The hardening recommendations in Section 8 map directly to the weaknesses this specific attack chain would have exploited.

This project took me through a simulated end to end: the attached email in Section 4 shows clearly, multifactor evidence of phishing(failures of the authentication, spoofing domain, contents that reveal social engineering); the PDF attached in Section 5 has a structural indicators which is consistent with an auto-executing, payload-carrying malicious documents, recorded by hash for future detection; and in section 6 I have demonstrated a concrete, working mitigation-encryption at rest with time-limited. For a sensitive data that would be at risk if such a compromise succeeded. In section 8 the hardening recommendation that directly map to the weaknesses of the specific attack chain was explained.